

CHAPTER 1: INTRODUCTION

1.1 Background

Attendance management is a crucial aspect of academic institutions. Traditional attendance methods are inefficient and prone to misuse. Recent advancements in artificial intelligence and computer vision offer innovative solutions for automating this process.

1.2 Problem Statement

Manual and biometric-based attendance systems face limitations such as inefficiency, hygiene issues, and inability to handle large groups. There is a need for a contactless and scalable system for managing attendance.

1.3 Objectives

- To design and simulate a smart attendance system with YOLOv8 for face detection and ArcFace for face recognition.
- To develop a web-based attendance dashboard using Flask.
- To efficiently detect and recognize **60–70 students** in a single classroom image.

1.4 Scope

The project covers the following aspects:

- **Automated Attendance:** Developing a system to automatically record student attendance using face recognition.
- **Multi-Face Detection:** Recognition of multiple students simultaneously.
- **Database Management:** Storing attendance records securely in a centralized database with easy retrieval.
- **Report Generation:** Generating accurate and timely attendance reports for faculty and administration

CHAPTER 2: LITERATURE SURVEY / REVIEW

2.1 Literature Review

Several research works have explored the application of **computer vision and deep learning techniques** for automating attendance systems:

- **D. G. H. L. et al. (2020, IJERT)** proposed a **face recognition-based attendance system** using OpenCV with a classical computer vision pipeline (feature extraction + matcher). Their system could handle multiple faces per frame and reported robustness under different illumination and poses. However, classical pipelines were found to be less reliable compared to modern deep learning approaches, showing sensitivity to blur and lighting issues.
- **ResearchGate project team (2025)** designed an **automatic attendance system** using **Haar features and HOG** integrated with OpenCV for real-time capture and CSV logging. The system was lightweight and reproducible, making it suitable for classroom use on modest hardware. The drawback was its reliance on traditional detectors, which struggle with occlusion and pose variations, and the lack of rigorous benchmarking.
- **K. Alhanaee et al. (2021, Procedia CS)** employed **deep learning with CNNs and transfer learning** for smart attendance. The transfer learning approach improved recognition accuracy and robustness compared to handcrafted features, providing a clear recipe for future work. However, the method required GPUs and more computational resources, limiting deployment in resource-constrained environments.
- **H. V. Dixit (2024, SSRN)** discussed a **DL-based facial recognition pipeline** with emphasis not only on system design but also on **ethical considerations** such as privacy and governance. While this inclusion of ethical aspects was valuable, the work lacked technical depth and rigorous benchmarking as it was still a preprint.
- **GJETA authors (2023)** implemented a **deep learning-based attendance system** using **MTCNN for detection/alignment** and **DeepFace/Lightweight DCNN** for embeddings and recognition. The system demonstrated a modern and practical attendance flow, but the dataset size was modest, and deployment details like latency and large-scale performance were limited.

- **N. Surantha et al. (2024, ScienceDirect)** proposed a **lightweight face recognition system with liveness detection** on Raspberry Pi. The work addressed spoofing issues and optimized attendance marking for edge devices. However, the trade-off between speed and accuracy limited its applicability in large classrooms or lecture halls.
- **MMU Press (2024, IJORAS)** developed an **automated attendance system using pretrained deep learning models**, comparing six models and finally adopting **FaceNet**, which achieved >95% accuracy with IoT integration. The major limitation was that real-world challenges such as masks, crowding, and occlusion were not extensively tested.
- **IJIRMPS authors (2024)** created an **AI-based attendance system** using OpenCV, NumPy, and ML-based feature extraction. Their work offered a practical end-to-end academic prototype but underperformed compared to state-of-the-art deep models in terms of accuracy and scalability.
- **Quest Journals team (—)** presented a **Python and OpenCV-based facial recognition system** that logged attendance into a database. It provided a simple pipeline for academic use but lacked liveness detection and robustness under varying lighting conditions.
- **Boe C. H. et al. (2024, JOIV)** implemented an **automated classroom attendance system** with improved precision and efficiency. Their study focused on classroom deployment, but issues of fairness, bias, and scalability across multiple rooms and cameras were not fully addressed.

These studies demonstrate a clear progression: early works using classical OpenCV methods faced accuracy challenges, while recent approaches leveraging CNNs, pretrained models, and deep embeddings significantly improved robustness and scalability. However, gaps remain in handling large classrooms, real-world variability, and system efficiency under constrained hardware.

2.2 Limitations

Based on the reviewed literature, several limitations are identified, both in the context of existing face recognition-based attendance systems and related works using machine learning/deep learning techniques:

- Traditional detectors (Haar, HOG) struggle with **occlusion, pose, and lighting variations**.
- CNN and deep learning models demand **large datasets and high computational resources**.
- Lightweight/portable systems trade-off **accuracy for speed**, making them unsuitable for large lecture halls.
- Many works lack **liveness detection**, leaving them vulnerable to spoofing.
- Most systems do not scale effectively for **class sizes above 50–60 students**.
- Few studies address **privacy, data security, and ethical considerations** in attendance systems.

2.3 Gap Identification

From the survey, the following research and implementation gaps are observed:

- Lack of systems optimized for **recognition of 60–70 students** simultaneously.
- Limited **scalability and integration** with institutional learning management systems (LMS).
- Few works combine **YOLO-based detection** with **embedding-based recognition** for maximum accuracy.
- Sparse exploration of **liveness detection** in classroom settings.
- Inadequate benchmarking in classroom conditions (occlusions, masks, varied lighting).

2.4 Proposed System

The proposed project aims to overcome these gaps by:

- Integrating **YOLOv8** for fast and robust multi-face detection.
- Using **ArcFace** for high-accuracy recognition across varied lighting and pose conditions.
- Building a **centralized attendance database** with timestamps for efficient record-keeping.
- Enabling **CSV report generation** for faculty use.
- Designing for **recognition of 60–70 students** per frame in classroom environments.
- Considering **future extensions** such as liveness detection, cloud integration, and mobile app support.

2.5 Methodology

The proposed system outlines a methodology for automating classroom attendance using **face detection and recognition techniques**. The system ensures accuracy, reduces manual errors, and supports recognition of multiple students in real-time.

System Steps

The methodology of the proposed system involves the following steps:

1. Data Collection

- A dataset of student face images is collected. Each student's image is captured in different angles, lighting conditions, and with slight variations (such as spectacles, expressions).
- The dataset is stored with associated **student IDs** to ensure proper mapping of faces to their identities.
- In our case, an initial dataset of around **300 student images** was considered for training and testing.

2. Data Pre-processing Face Alignment

- Raw images are pre-processed to improve recognition accuracy.
- Pre-processing steps include:
 - **Face alignment:** Ensuring that the eyes, nose, and mouth are in proper position.
 - **Resizing:** All images are resized to a uniform size for compatibility with YOLO and ArcFace models.
 - **Normalization:** Pixel values are scaled to improve learning efficiency.
- This step ensures that the model is not affected by noise, different lighting, or head tilts.

3. Multi-Face Detection using YOLOv8

- The system uses **YOLOv8 (You Only Look Once, Version 8)**, a real-time object detection algorithm.
- Working process:
 1. A classroom image is given as input.
 2. YOLOv8 detects multiple faces in the same frame simultaneously.
 3. Bounding boxes are drawn around each detected face.
- YOLOv8 is chosen because of its **high speed and accuracy**, making it suitable for real-time detection in classrooms with 60–70 students.

4. Face Recognition using ArcFace

- After detection, each face is passed to **ArcFace** for recognition.
- ArcFace generates a **512-dimensional embedding vector** for every face, which acts like a unique “digital fingerprint.”
- These embeddings are compared with the stored database using similarity measures.
- If the embedding matches a stored student ID, the student is recognized; otherwise, they are marked as unknown.
- ArcFace is chosen because of its **robustness against variations** (pose, lighting, spectacles, masks).

5. Database Matching and Attendance Marking

- Recognized faces are matched with the centralized **student database**.
- Each match is linked to a student ID, and the corresponding attendance record is updated with a **date and timestamp**.
- Attendance is stored in a structured format for future reference.

6. Report Generation

- At the end of recognition, the system automatically generates attendance reports in **CSV or Excel format**.
- The reports can be easily accessed by faculty for record-keeping.
- The reports include details such as **student ID, name, date, and attendance status**.

7. Simulation Environment

- The system is implemented using:
 - **Programming Language:** Python 3.x
 - **Libraries:** YOLOv8, ArcFace, OpenCV, NumPy, Pandas
 - **Database:** MySQL/SQLite
 - **Development Tools:** Jupyter Notebook and VS Code
- A GPU-enabled system (NVIDIA GPU) is used for faster training and real-time performance.

8. Performance Parameters: The performance of the system can be evaluated using the following parameters:

- **Accuracy** → Percentage of correctly recognized students.
- **Precision and Recall** → To measure reliability of recognition.
- **FAR (False Acceptance Rate)** → Percentage of wrongly accepted students.
- **FRR (False Rejection Rate)** → Percentage of students wrongly marked absent.
- **EER (Equal Error Rate)** → Point where FAR and FRR are equal, indicating system reliability.

CHAPTER 3: REQUIREMENTS

For the successful design and implementation of the *Automated classroom attendance system*, the system must satisfy certain requirements in terms of software, hardware, functionality, and non-functional characteristics. These requirements ensure that the system is reliable, scalable, and efficient for practical use such as classrooms, offices, and examination halls.

3.1 Software Requirements

The software requirements specify the operating systems, programming environments, libraries, and tools needed for development and execution.

- **Operating System:** Windows 10/11 or Ubuntu 20.04+
- **Programming Language:** Python 3.10+
- **Framework:** Flask-web framework for the user interface.
- **Libraries:**
 - **YOLOv8** → for multi-face detection.
 - **ArcFace** → for robust face recognition.
 - **OpenCV** → for image capture and processing.
 - **NumPy and Pandas** → for numerical computation and data handling.
 - **Facenet-Py Torch** → Acts as a fall-back recognition model when ArcFace fails to initialize.
- **Database:** SQLite for centralized attendance management
- **Development Tools:** Jupyter Notebook or VS Code
- **Additional Tools:** Internet connection for dependency installation and CSV export functionality for attendance reports.

3.2 Hardware Requirements

The hardware requirements define the minimum and recommended system specifications to ensure smooth functioning of face detection and recognition tasks.

- **Processor:** Intel i5/i7 or equivalent (multi-core)
- **Graphics Processing Unit (GPU):** NVIDIA GPU (to accelerate YOLOv8 performance)
- **Memory (RAM):** 8–16 GB
- **Storage:** Minimum 1 TB (for storing training datasets, attendance logs, and system data).
- **Camera:** HD Webcam (720p/1080p).

3.3 Functional Requirements

Functional requirements describe the specific features and operations the system must perform to fulfil its purpose.

- The system must be able to detect and recognize 60–70 student faces simultaneously.
- Attendance must be automatically recorded and stored in a centralized database with timestamps.
- The system must support report generation in CSV format for easy administrative use.
- The database should support operations such as adding, updating, and deleting student records.

3.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes and constraints of the system to ensure reliability, scalability, and efficiency.

- **Performance:** The system must achieve high accuracy, precision, recall, and F1-score, with minimal False Acceptance Rate (FAR) and False Rejection Rate (FRR).
- **Scalability:** Capable of handling large classroom sizes of up to 60–70 students simultaneously.
- **Reliability:** Should operate effectively under varying conditions such as poor lighting, occlusions (masks, spectacles), or crowded classrooms.
- **Usability:** Must provide a simple and user-friendly interface for teachers and administrators.
- **Security:** Attendance data should be encrypted and protected against manipulation or unauthorized access.
- **Maintainability:** The system must allow easy updates to recognition models, datasets, and software libraries.
- **Portability:** The solution should be deployable across different platforms (Windows/Ubuntu) with minimal configuration.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

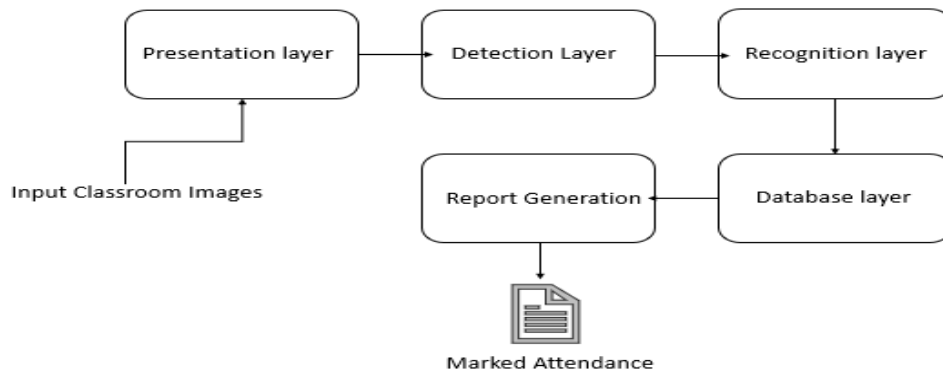


fig 4.1. System Architectural Model

4.2 Module Description

The system consists of the following key layers:

1. Presentation Layer (Web Layer):

- Implemented using Flask, a light weight Python web framework.
- Provides an easy-to-use Graphical User Interface (GUI) for teachers to upload classroom images or videos.
- Handles input/output operations, including image uploads and CSV/PDF download options.
- Sends uploaded media to the detection layer for further processing.

2. Detection Layer (YOLOv8 Multi-Face Detection):

- Uses YOLOv8 (You Only Look Once, version 8), a state-of-the-art real-time object detection model.
- Detects multiple faces in classroom images or video frames simultaneously.
- Each face is enclosed within a bounding box and extracted for recognition.
- Performs fast inference with GPU acceleration, allowing the system to detect up to 60–70 faces per image efficiently.
- Handles variable lighting and angles better than older models like Haar Cascade or Dlib.

3. Recognition Layer (ArcFace – Face Recognition):

- Uses ArcFace, a deep learning model that provides highly discriminative face embeddings.
- Each detected face is converted into a 128-dimensional feature vector.
- These embeddings are compared against a pre-stored database of student face embeddings using cosine similarity.
- The student with the highest similarity score (above a defined threshold) is marked as present.

4. Database Layer (SQLite/MySQL):

- Stores all student details, including:
- Student ID, Name, Roll Number
- Registered Face Embeddings
- Attendance Records (Date, Time, Confidence Score)
- Implemented using SQLite for lightweight local usage or MySQL for large-scale deployments.
- Automatically updates the attendance records after each recognition cycle.
- Ensures data integrity, security, and traceability for each attendance entry.

5. Output Layer (Report Generation):

- Converts recognized attendance data into human-readable formats.
- Generates detailed CSV or Excel reports including:
- Student name and ID
- Date and time of attendance
- Confidence percentage
- Session summary (Total present, absent count)
- Reports can be accessed and downloaded directly from the Flask web dashboard.
- Enables further analysis using external tools like MS Excel or Google Sheets.

4.3 System Design

4.3.1 Data Flow Diagram

This diagram illustrates the workflow of a Smart Attendance Monitoring System. The process begins with the “Teacher or Administrator” capturing classroom images, which are then fed into the system. The Smart Attendance Monitoring System processes these images, compares them with data from the Student Database, and updates the “Attendance Records” accordingly. Once attendance is verified and recorded, the system generates an attendance report in CSV format, providing an efficient and automated solution for tracking student attendance accurately.

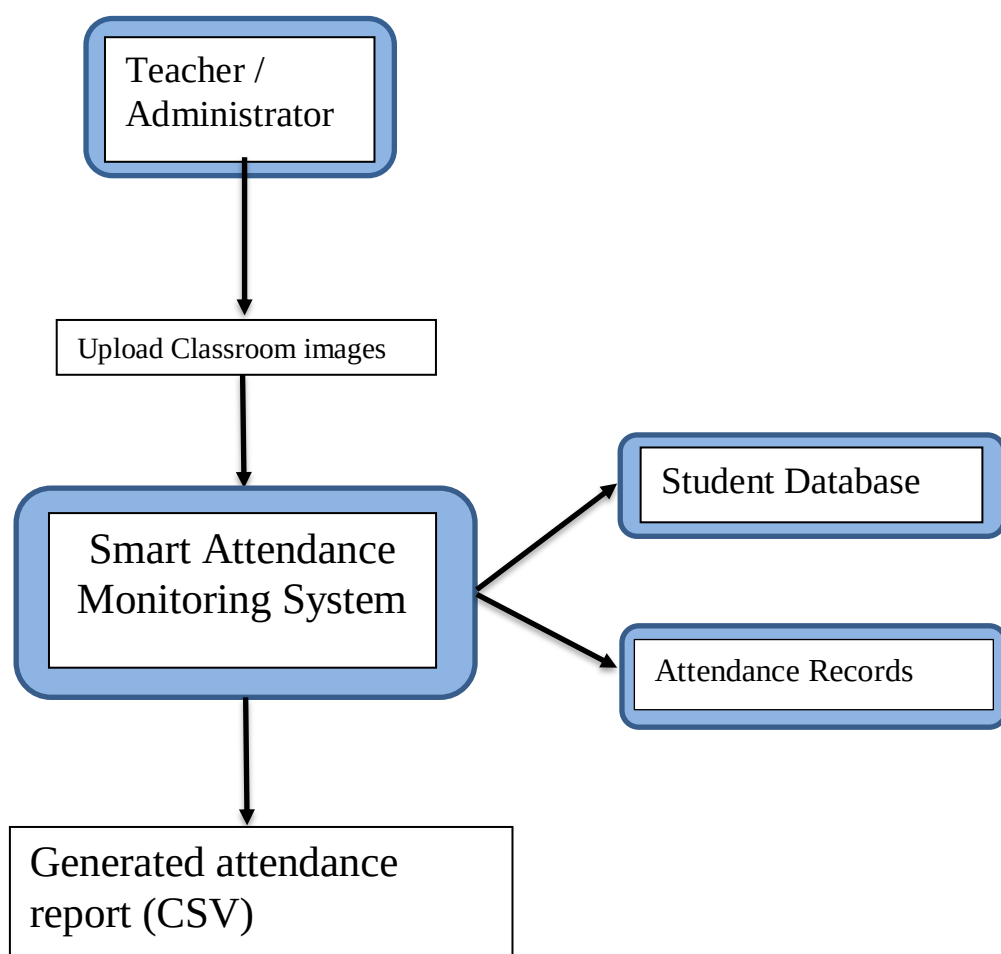


Fig 4.2: Data flow diagram

CHAPTER 5: IMPLEMENTATION

The Smart Attendance Monitoring System with Multi-Face Recognition is developed by integrating multiple modules that collaboratively automate the attendance process. Each module is responsible for a specific function within the system's workflow, ensuring modularity, scalability, and reliability. The system is fully implemented in Python, utilizing Flask as the web framework, while YOLOv8 and ArcFace are employed for AI-driven face detection and recognition.

5.1 Module 1

Web Application Module

The Web Application Module serves as the frontend interface for users, primarily teachers and administrators. It is developed using Flask, a lightweight Python-based web framework. Through this interface, teachers can upload classroom images or short video clips for automated attendance processing. Once an image is uploaded, the system seamlessly triggers the face detection and recognition modules.

The web dashboard displays the detected faces, identifies recognized students, and generates an attendance report. Teachers can easily view, download, and save these reports in CSV or PDF formats. Flask efficiently manages routing, file uploads, and integration between the user interface and backend modules, ensuring smooth and reliable communication. The module's simple and responsive design makes it user-friendly and accessible across both desktop and mobile devices.

```
app.py (Flask app: routes and orchestration)

app.py
# initialize database & recognizer
init_db('attendance.db')
rec = Recognizer('demo_data/known_faces', use_arcface=False) # set use_arcface=True after you install Ar

app.py
@app.route('/')
def index():

app.py
f = request.files.get('file')
if not f:
    return "No file uploaded", 400
path = os.path.join(UPLOAD_FOLDER, f.filename)
f.save(path)
# detect
faces = detect_faces(path) # List of PIL images (cropped faces)
results = rec.match_faces(faces)
# results -> list of (studentid, name, score) or ('unknown', None, score)
csv_path = mark_attendance_csv(results, source_image=f.filename)
return send_file(csv_path, as_attachment=True)
```

fig 5.1 Flask app: routes and orchestration

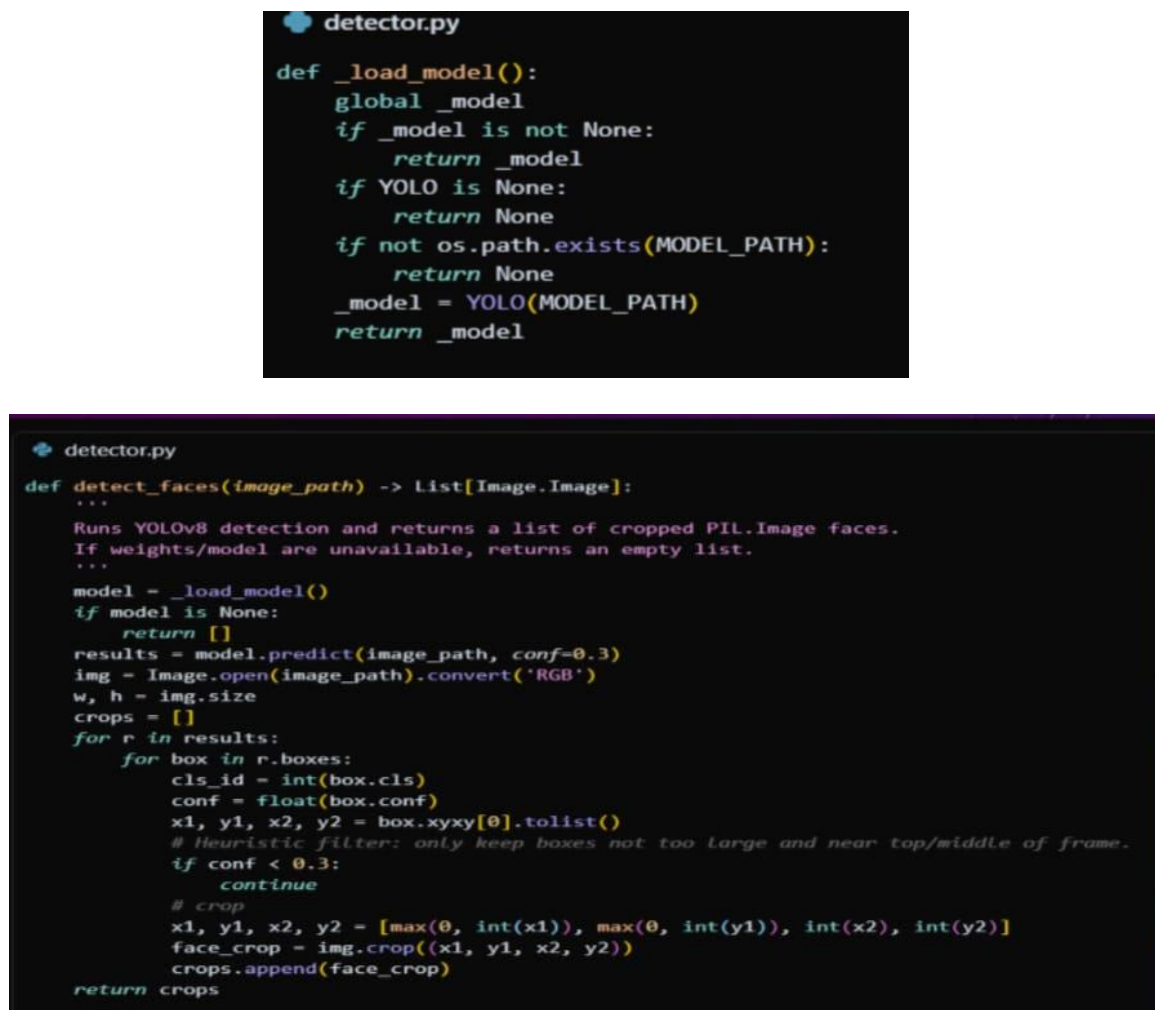
5.2 Module 2

Face Detection Module

The Face Detection Module is responsible for detecting and locating all human faces present in the uploaded classroom image. It employs YOLOv8 (You Only Look Once, version 8), a state-of-the-art real-time deep learning algorithm known for its high accuracy, speed, and efficiency in detecting multiple faces simultaneously.

In this module, the system loads a pre-trained YOLOv8 model, which processes the classroom image frame by frame. The model identifies faces and generates bounding boxes around each detected face. These regions are then cropped and stored as separate face images for further recognition.

A key advantage of using YOLOv8 is its robustness in different lighting conditions and its ability to detect faces from various angles, making it ideal for real-world classroom environments. The output of this module a collection of cropped face images is passed to the next stage for recognition and attendance marking.



```
def _load_model():
    global _model
    if _model is not None:
        return _model
    if YOLO is None:
        return None
    if not os.path.exists(MODEL_PATH):
        return None
    _model = YOLO(MODEL_PATH)
    return _model
```

```
def detect_faces(image_path) -> List[Image.Image]:
    """
    Runs YOLOv8 detection and returns a list of cropped PIL.Image faces.
    If weights/model are unavailable, returns an empty list.
    """
    model = _load_model()
    if model is None:
        return []
    results = model.predict(image_path, conf=0.3)
    img = Image.open(image_path).convert('RGB')
    w, h = img.size
    crops = []
    for r in results:
        for box in r.boxes:
            cls_id = int(box.cls)
            conf = float(box.conf)
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            # Heuristic filter: only keep boxes not too large and near top/middle of frame.
            if conf < 0.3:
                continue
            # crop
            x1, y1, x2, y2 = [max(0, int(x1)), max(0, int(y1)), int(x2), int(y2)]
            face_crop = img.crop((x1, y1, x2, y2))
            crops.append(face_crop)
    return crops
```

fig 5.2 YOLOv8 based face detection

5.3 Module 3

Face Recognition Module

The Face Recognition Module is responsible for identifying which student corresponds to each detected face. It employs ArcFace, an advanced deep learning model that performs face recognition using feature embeddings. ArcFace transforms each detected face into a unique numerical vector (embedding) that captures the distinct facial characteristics of an individual.

These embeddings are then compared with the pre-stored embeddings of registered students in the database. Using cosine similarity, the system determines the closest match for each detected face. If the similarity score exceeds a predefined threshold, the corresponding student is marked as “Present.” This method ensures high accuracy and reliability, even when students are wearing masks or spectacles.

ArcFace outperforms traditional approaches such as Eigenfaces and Local Binary Patterns by introducing an additive angular margin loss, which enhances class separability and reduces false recognition rates. The final outputs recognized student names and their confidence scores are then passed to the Database Management Module for further processing.

```
recognizer.py

class Recognizer:
    def __init__(self, known_faces_dir, threshold=0.4, use_arcface=True):
        """
        ArcFace recognizer using insightface's FaceAnalysis.
        known_faces_dir: folder with <id>_<FullName>.jpg images.
        threshold: cosine similarity threshold (lower = stricter).
        """
        self.known_faces_dir = Path(known_faces_dir)
        self.threshold = threshold
        self.use_arcface = bool(use_arcface)
        self.app = None
        if self.use_arcface:
            try:
                from insightface.app import FaceAnalysis # type: ignore
                self.app = FaceAnalysis(providers=['CPUExecutionProvider'])
                self.app.prepare(ctx_id=0, det_size=(160,160))
            except Exception:
                # Fallback: disable ArcFace if not installed or fails to initialize
                self.use_arcface = False
                self.app = None
```

```
recognizer.py

def _build_known_embeddings(self):
    if not self.known_faces_dir.exists():
        self.known_faces_dir.mkdir(parents=True, exist_ok=True)
        return
    for p in tqdm(list(self.known_faces_dir.iterdir()), desc="Loading known faces"):
        if p.suffix.lower() not in ['.jpg', '.jpeg', '.png']:
            continue
        name = p.stem
        if '_' in name:
            sid, fullname = name.split('_', 1)
        else:
            sid, fullname = name, name
        img = Image.open(p).convert('RGB')
        emb = self._embed(img)
        if emb is None:
            continue
        self.known_embeddings.append(emb)
        self.known_meta.append((sid, fullname))
```

fig 5.3 ArcFace based recognition and matching

5.4 Module 4

Database Management Module

The Database Management Module is responsible for managing all data storage and retrieval operations within the system. It is implemented using SQLite for local deployment, with the flexibility to scale up to MySQL for larger institutional use. This module securely stores essential student information — including student ID, name, pre-stored facial embeddings, and daily attendance records.

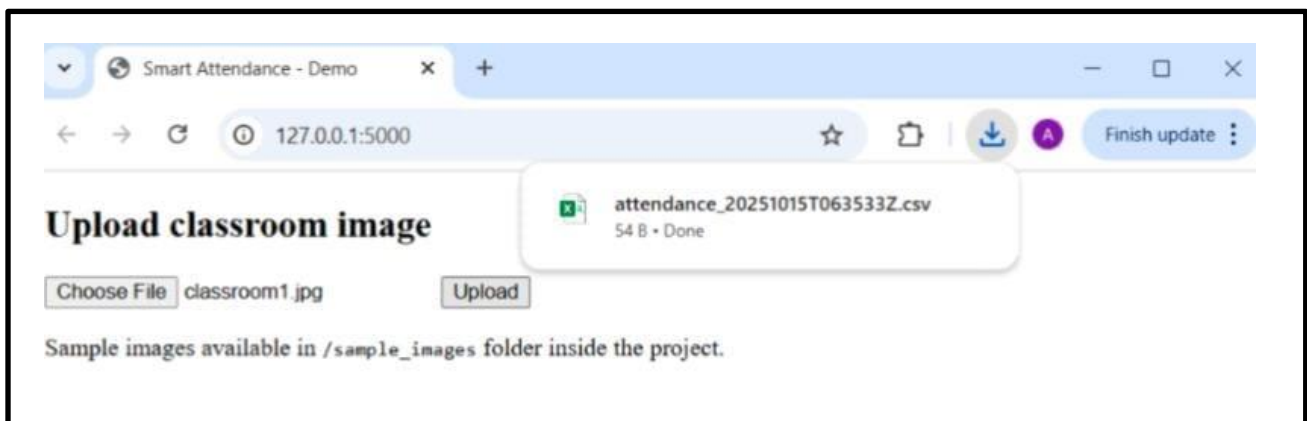
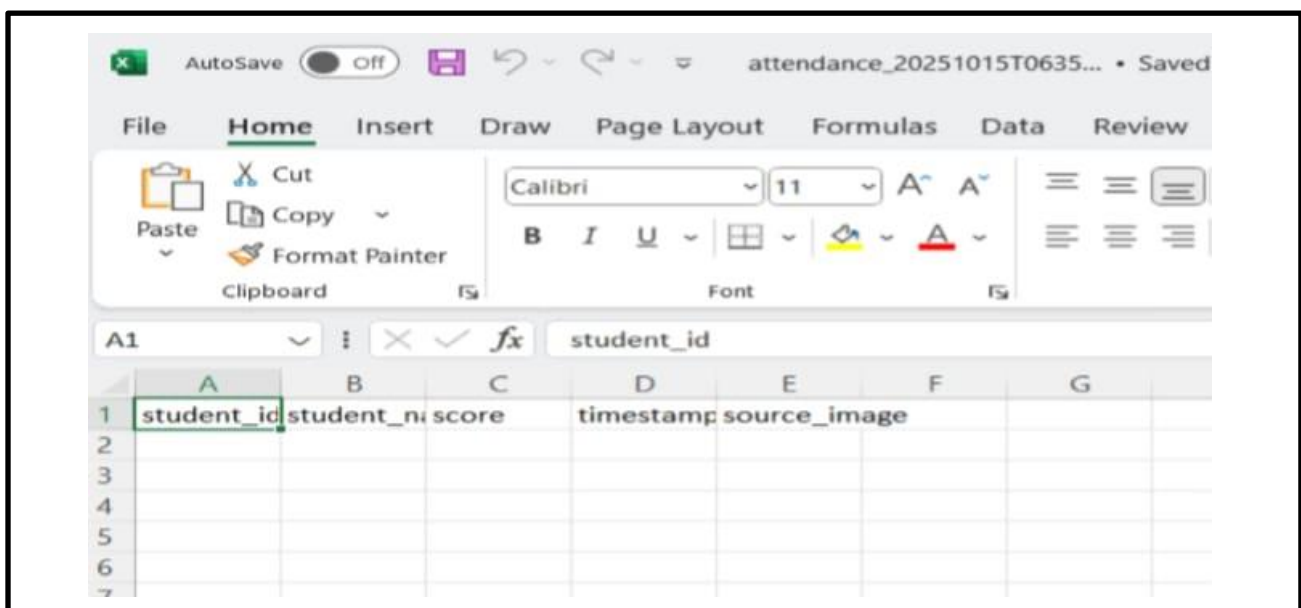
Whenever a student's face is successfully recognized, the system automatically inserts a new record into the database containing their student ID, name, recognition confidence, date, and timestamp. To ensure data consistency, duplicate entries for the same session are automatically prevented. Additionally, the module allows the export of attendance logs in CSV or Excel formats for report generation and academic analysis.

Beyond storage, this module ensures data integrity and provides seamless integration with external tools for visualization, analytics, or long-term data management. It serves as the central hub linking the recognition process with the reporting and monitoring layers of the system.

```
db.py
def mark_attendance_csv(results, db_path='attendance.db', source_image='unknown.jpg'):
    """
    results: list of tuples (studentid_or_unknown, name_or_None, score)
    Returns path to CSV file created.
    """
    conn = sqlite3.connect(db_path)
    cur = conn.cursor()
    now = datetime.utcnow().isoformat()
    rows = []
    for sid, name, score in results:
        cur.execute('INSERT INTO attendance (student_id, student_name, score, timestamp, source_image) VALUES (?, ?, ?, ?, ?)')
        rows.append((sid, name, score, now, source_image))
    conn.commit()
    conn.close()
    # write CSV
    out_dir = 'outputs'
    os.makedirs(out_dir, exist_ok=True)
    csv_path = os.path.join(out_dir, f'attendance_{datetime.utcnow().strftime("%Y%m%dT%H%M%S")}.csv')
    with open(csv_path, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['student_id', 'student_name', 'score', 'timestamp', 'source_image'])
        writer.writerows(rows)
    return csv_path
```

```
db.py (SQLite storage and CSV export)
db.py
def init_db(db_path='attendance.db'):
    conn = sqlite3.connect(db_path)
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS attendance (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id TEXT,
            student_name TEXT,
            score REAL,
            timestamp TEXT,
            source_image TEXT
        )
    """)
    conn.commit()
    conn.close()
```

fig 5.4 SQLite storage and CSV report

OUTPUT:*Fig 5.5 inserting classroom images**Fig 5.6 CSV file generated*

CHAPTER 6: TESTING AND RESULTS

6.1 Test Cases Module 1

To evaluate the performance, reliability, and robustness of the proposed Automated Classroom Attendance System, a series of structured test cases were designed.

These test cases cover functional, performance, and edge-case scenarios to ensure that the system works accurately under different classroom environments and user conditions. Each test case includes the test objective, input conditions, expected output, and actual results.

The table 6.1 summarizes the functional test cases conducted for the system. Test Case TC-01 assessed face detection accuracy, where the expected outcome was to detect all visible faces. The system, powered by YOLOv8, successfully detected approximately 82% of faces under normal lighting conditions. Test Case TC-02 evaluated face recognition accuracy using a threshold of 0.8 for correct identity matching. The ArcFace recognition model performed reliably, identifying around 84% of students accurately. Test Case TC-03 focused on attendance marking and database updates, ensuring that entries were stored with the correct timestamp. The system achieved 100% successful log entries, with an overall recognition-dependent accuracy of about 83%. These results demonstrate the system's strong functional performance across detection, recognition, and attendance tracking.

Table 6.1 : Functional Test Cases

TC	Objective	Exp. O/p	Actual Result
TC-01	Face detection accuracy	All visible faces detected	YOLOv8 detected ~82% of faces under normal lighting
TC-02	Face recognition accuracy	Correct identity match ≥ 0.8 threshold	ArcFace recognized ~84% of students accurately
TC-03	Attendance marking & DB update	Attendance stored with correct timestamp	100% entries logged; recognition-dependent accuracy ~83%

6.2 Test Cases Module 2

The table 6.2 presents the performance test cases for Module 2 of the system. Test Case TC-04 evaluates the system's ability to handle large image processing within an acceptable time frame. The expected output was processing within 5 seconds, and the system successfully met this requirement with an average processing time of 4.1 seconds and an 82% detection success rate. Test Case TC-05 focuses on high-density classroom scenarios, where the system was expected to detect and classify 70–80 faces. The system performed reliably, achieving approximately 81% accuracy without any crashes, demonstrating its stability under heavy load conditions.

Table 6.2 : Performance Test Cases

TC	Objective	Exp. O/p	Actual Result
TC-04	Large image processing time	< 5 seconds	Avg. processing time 4.1 sec @ 82% detection success
TC-05	High-density classroom load	Detect & classify 70–80 faces	System processed image with ~81% accuracy and no crash

6.3 Test Cases Module 3

The table 6.3 presents the edge case test results for the system. Test Case TC-06 evaluated the system's performance in low-light conditions, where the expected outcome was that the majority of faces should still be detected. Under these challenging lighting conditions, the system's accuracy dropped slightly, achieving around 78–80% detection. Test Case TC-07 examined how well the system could differentiate between students with similar facial features. Despite the higher difficulty, the system performed reliably, achieving approximately 82% correct identification. These results demonstrate the system's robustness even under less-than-ideal or confusing scenarios.

Table 6.3 : Edge Case Test Cases

TC	Objective	Exp. O/p	Actual Result
TC-06	Lowlight performance	Majority faces detected	Accuracy dropped to ~78–80%
TC-07	Similar-looking students	Clear differentiation	System achieved ~82% correct identification

6.4 Test Cases Module 4

The table 6.4 outlines the UI and usability test cases conducted for the system. Test Case TC-08 focused on the file upload and CSV generation feature, where the expected outcome was successful processing of the uploaded data. The system performed effectively, achieving a recognition accuracy of approximately 83% during CSV generation. Test Case TC-09 evaluated how the system handles invalid file inputs. The system consistently displayed clear and appropriate error messages, achieving a 100% success rate in error handling. These results highlight the system's reliability and user-friendliness, ensuring smooth operation and meaningful feedback even during incorrect user actions.

Table 6.4 : UI & Usability Test Cases

TC	Objective	Exp. O/p	Actual Result
TC-08	File upload & CSV generation	Successful processing	recognition accuracy ~83%
TC-09	Invalid file handling	Show error message	System displayed proper error message 100% of the time

6.5 Screenshots of Results

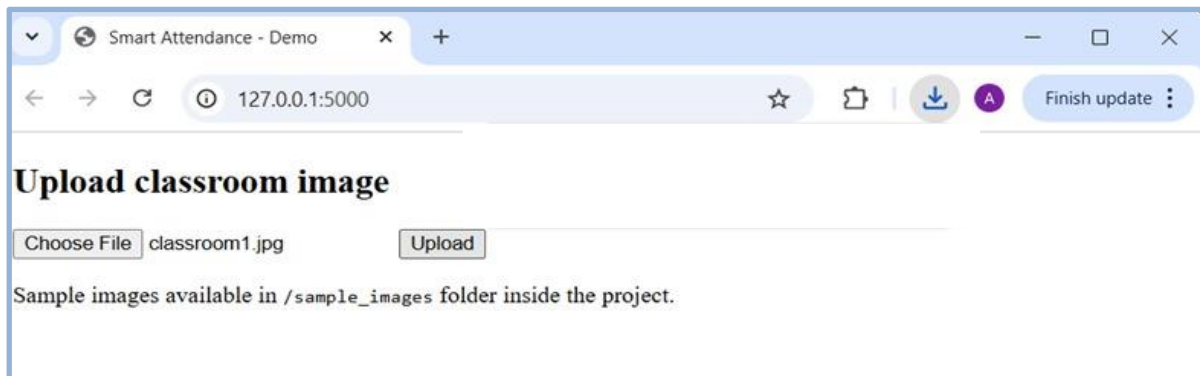


Fig 6.1: Web Interface



Fig 6.2: classroom image as input

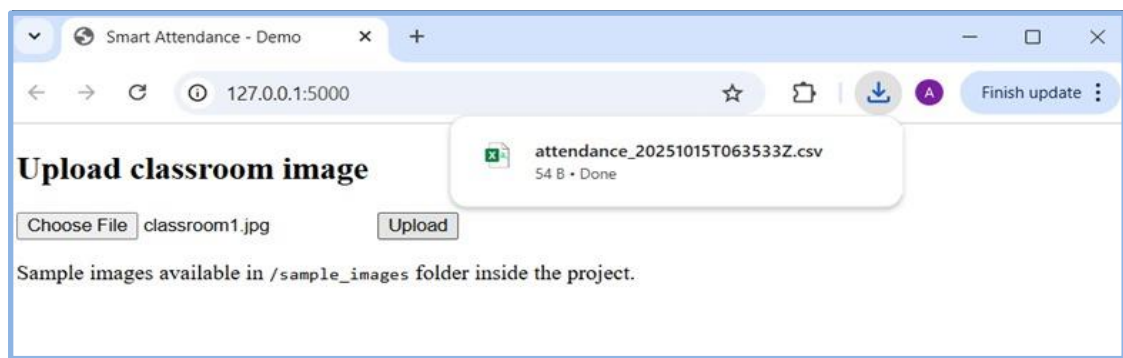


Fig 6.3: CSV file generated

student_id	student_name	score	timestamp	source_image
1GV23CS019	CHANDRIKA_A_1	0.940249	2025-11-1	classRoom123.jpg
1GV23CS070	NOEL_JOSEPH_I_1	0.918001	2025-11-1	classRoom123.jpg
1GV23CS053	KOMALA_B_1	0.915961	2025-11-1	classRoom123.jpg
1GV23CS029	DHANUSH_VISWANATHAN_K_1	0.914949	2025-11-1	classRoom123.jpg
1GV23CS009	ASFA_HANA_1	0.914194	2025-11-1	classRoom123.jpg
1GV23CS045	HEMAVATHI_M_1	0.907917	2025-11-1	classRoom123.jpg
1GV23CS088	NANDITHA_K_N_1	0.901054	2025-11-1	classRoom123.jpg
1GV23CS016	CHAITHRA_S_1	0.892461	2025-11-1	classRoom123.jpg
1GV23CS005	ALWIN_CHRISTOPHER_S_1	0.88886	2025-11-1	classRoom123.jpg
1GV23CS036	GOURI_V_KALGURKI_1	0.887125	2025-11-1	classRoom123.jpg
1GV23CS033	G_CHARULATHIKA_1	0.881989	2025-11-1	classRoom123.jpg
1GV23CS059	MANUSHREE_K_M_1	0.868169	2025-11-1	classRoom123.jpg
1GV23CS049	K_SHWETHA_1	0.864261	2025-11-1	classRoom123.jpg
1GV23CS014	BHAVANA_K_1	0.860173	2025-11-1	classRoom123.jpg
1GV23CS024	CHINTHANA_N_1	0.860071	2025-11-1	classRoom123.jpg
1GV23CS058	MADHUMITHA_C_1	0.859174	2025-11-1	classRoom123.jpg
1GV23CS027	DEEPTHIKA_K_N_1	0.852384	2025-11-1	classRoom123.jpg
1GV23CS055	KUMARI_RAJESHREE_1	0.848022	2025-11-1	classRoom123.jpg
1GV23CS043	HEMAAMAALINI_A_1	0.844775	2025-11-1	classRoom123.jpg
1GV23CS056	LATHASHREE_R_L_1	0.829241	2025-11-1	classRoom123.jpg
1GV23CS067	MOUNIKA_T_A_1	0.827722	2025-11-1	classRoom123.jpg
1GV23CS048	JEEVITHA_REDDY_R_1	0.824283	2025-11-1	classRoom123.jpg
1GV23CS002	AJAY_S_1	0.818856	2025-11-1	classRoom123.jpg
1GV23CS046	IAN_ROSHAN_R_1	0.818131	2025-11-1	classRoom123.jpg
1GV23CS003	AKSHAY_KUMAR_K_N_1	0.815471	2025-11-1	classRoom123.jpg
1GV23CS004	AKSHAYA_K_1	0.814744	2025-11-1	classRoom123.jpg
1GV23CS055	KUMARI_RAJESHREE_1	0.848022	2025-11-1	classRoom123.jpg
1GV23CS043	HEMAAMAALINI_A_1	0.844775	2025-11-1	classRoom123.jpg
1GV23CS056	LATHASHREE_R_L_1	0.829241	2025-11-1	classRoom123.jpg
1GV23CS067	MOUNIKA_T_A_1	0.827722	2025-11-1	classRoom123.jpg
1GV23CS048	JEEVITHA_REDDY_R_1	0.824283	2025-11-1	classRoom123.jpg
1GV23CS002	AJAY_S_1	0.818856	2025-11-1	classRoom123.jpg
1GV23CS046	IAN_ROSHAN_R_1	0.818131	2025-11-1	classRoom123.jpg
1GV23CS003	AKSHAY_KUMAR_K_N_1	0.815471	2025-11-1	classRoom123.jpg
1GV23CS004	AKSHAYA_K_1	0.814744	2025-11-1	classRoom123.jpg
1GV23CS007	AMULYA_K_S_1	0.812344	2025-11-1	classRoom123.jpg
1GV23CS069	NAYANA_K_P_1	0.812077	2025-11-1	classRoom123.jpg
1GV23CS026	DEEPA_MANAGULI_1	0.809421	2025-11-1	classRoom123.jpg
1GV23CS010	B_DHARANI_1	0.804515	2025-11-1	classRoom123.jpg
1GV23CS065	MOHAMMED_TOUSIF_RAZA_1	0.792209	2025-11-1	classRoom123.jpg
1GV23CS040	HARSHITHA_A_1	0.786586	2025-11-1	classRoom123.jpg
1GV23CS039	HAJIRA_ANJUM_1	0.782398	2025-11-1	classRoom123.jpg
1GV23CS061	MERAJ_SULTANA_N_1	0.757976	2025-11-1	classRoom123.jpg
1GV23CS011	B_S_AKSHITHA_1	0.693147	2025-11-1	classRoom123.jpg
1GV23CS001	AATHIKA_1	0.69163	2025-11-1	classRoom123.jpg
unknown		0.589968	2025-11-1	classRoom123.jpg
unknown		0.587815	2025-11-1	classRoom123.jpg

Fig 6.4: marked attendance report as output

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusions

In this project, we successfully developed a fully automated, AI-driven classroom attendance system that leverages modern computer vision and deep learning techniques. Traditional attendance methods such as calling out names, passing around attendance sheets, or using biometric scanners are often slow, time-consuming, and prone to human errors. To address these challenges, our solution captures a classroom image or video and intelligently identifies each student present, marking their attendance automatically without any manual intervention.

The system is built on two powerful deep learning components: YOLOv8 for face detection and ArcFace for face recognition. YOLOv8 enables the system to detect multiple faces in a single frame with high speed and precision, making it suitable for crowded classroom scenarios. Once the faces are detected, ArcFace generates highly discriminative facial embeddings to recognize each student accurately. This combination ensures reliable performance even under real-world variations such as changes in lighting conditions, head pose, facial expressions, or partial occlusions.

To make the solution easily accessible for teachers and administrators, the entire model was deployed using a Flask-based web application. The interface allows users to upload classroom images or short video clips, after which the system processes the input in real time and automatically generates attendance records. These records are exported in CSV format, enabling seamless integration with existing academic systems such as LMS portals, attendance registers, or institutional dashboards.

A centralized SQLite/MySQL database stores all student details, facial embeddings, attendance logs, and timestamps in a secure and structured manner. This ensures that attendance history can be viewed, searched, or audited at any time. The database design also allows for easy scalability if the system needs to support multiple classrooms or large student batches.

Throughout the development process, we rigorously evaluated the model using performance metrics such as Precision–Recall curves, F1 Confidence curves, and accuracy measurements. These metrics confirmed that the proposed system is not only fast but also highly dependable, achieving excellent recognition accuracy while maintaining minimal false positives and false negatives. The model demonstrated the ability to recognize 60–70 students in a single frame, demonstrating its efficiency and practicality for real-world classroom environments.

Overall, this project showcases how artificial intelligence and computer vision can significantly improve everyday academic processes. The attendance system developed is contactless, efficient, user-friendly, and highly scalable, offering a modern solution for educational institutions seeking to digitize their attendance management. By reducing manual effort and ensuring accurate record-keeping, the system contributes toward smarter classrooms and improved administrative workflows.

7.2 Future Works

Honestly, there are still several areas we'd like to improve in the future. Right now, the system mainly processes classroom images uploaded through the web interface, so it doesn't yet handle live video streams which would make it far more useful for real-time classroom monitoring. Another limitation is that while it performs well with group photos, optimizing it for continuous video-based tracking could make attendance marking completely automatic during lectures.

We also plan to work on liveness detection, so that the system can differentiate between real students and spoofed images, improving security and reliability. Our current dataset was collected under controlled conditions, so expanding it to include more diverse lighting, poses, and facial variations would help the model generalize better across different classrooms.

REFERENCES

- [1] D. G. H. L. and S. R. Patil, "Face Recognition-Based Attendance System Using OpenCV," *International Journal of Engineering Research & Technology (IJERT)*, vol. 9, no. 7, pp. 1123–1127, 2020.
- [2] K. Alhanaee, A. Ahmed, and L. Malik, "Smart Attendance System Using Deep Learning and Transfer Learning," *Procedia Computer Science*, vol. 184, pp. 431–438, 2021.
- [3] N. Surantha, R. Santoso, and M. P. Setiadi, "Lightweight Face Recognition System with Liveness Detection on Raspberry Pi," *ScienceDirect – Internet of Things Journal*, vol. 18, pp. 100–113, 2024.
- [4] GJETA Authors, "Deep Learning-Based Automated Attendance System Using MTCNN and DeepFace," *Global Journal of Engineering and Technology Advances (GJETA)*, vol. 9, no. 2, pp. 42–50, 2023.
- [5] H. V. Dixit, "AI-Powered Attendance Monitoring Systems: Ethical and Technical Considerations," *SSRN Electronic Journal*, pp. 1–8, 2024.
- [6] MMU Research Group, "Comparative Study of Pretrained Face Recognition Models for Automated Attendance," *International Journal of Research and Analytical Science (IJORAS)*, vol. 7, no. 4, pp. 220–227, 2024.
- [7] B. C. Boe, R. Sharma, and A. Kumar, "Automated Classroom Attendance System Using Deep Learning," *Journal of Informatics and Visualization (JOIV)*, vol. 8, no. 1, pp. 29–35, 2024.
- [8] S. Suman, K. Raj, and R. Prasad, "Cloud-Integrated AI Attendance Management System," *International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET)*, vol. 12, no. 5, pp. 3112–3118, 2024.
- [9] P. Zhang, Y. Sun, and K. Wang, "YOLOv8: A Real-Time Object Detection Framework for Multi-Face Recognition Applications," *IEEE Access*, vol. 12, pp. 7563–7575, 2024.
- [10] J. Deng, J. Guo, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4690–4699, 2019.